Artifact Promotion as a Control Model for Stable Cloud Deployment

Build on Target Machines vs. Build Once and Promote Artifacts

Vasili Gavrilov
Software Architect and Full-Stack Engineer

Abstract

Modern production deployment is both a technical process and a control process. The central release question should not be "Did we build the latest code on production?" but "Which approved artifact is deployed?" This article presents artifact promotion as a compact control model for stable cloud deployment. It compares two common approaches: building from source on each target machine, and building once in a controlled environment, then promoting the same artifact through deployment environments. The artifact-promotion model separates source code, build output, deployment approval, runtime configuration, and production execution. Environment-specific runtime data has two categories: true secrets, such as passwords, API tokens, and private keys; and operational configuration, such as database connection strings, SMTP hosts, internal service URLs, bucket names, service identifiers, and environment flags. Both are supplied at runtime from an environment-scoped configuration set, not stored in source control and not embedded into the artifact. This separation reduces operational chaos, limits the blast radius of source-control compromise, supports rollback and reproducibility, and provides the release evidence expected in regulated and high-assurance environments such as healthcare, finance, federal systems, and defense.

1 The Release Question

Many deployment discussions start with the wrong question:

> Did we build the latest code on production?

A better release question is:

> Which approved artifact is deployed?

This shift is small but important. "Latest code" is a moving target. An approved artifact is a concrete release object: it has a version, checksum, source revision, build time, approval record, deployment history, and rollback path. Developers, project managers, testers, auditors, operations teams, and stakeholders can all reason about the same object.

The release is not an informal statement that production built something from the repository. The release is the artifact selected for deployment.

2 Two Deployment Models

Environment names are organizational vocabulary. One company may use development, integration, QA, staging, user acceptance, pre-production, production, disaster recovery, or customer-specific replicas; another company may use different names or combine several roles into one environment. The deployment model should not depend on those names. It should answer a more basic question: where is the application built?

Approach A: build on target machines. Each deployment machine retrieves source code, resolves dependencies, runs the build locally, and deploys the locally produced output.

Approach B: build once and promote an artifact. One controlled build produces a deployable artifact. That same artifact is stored, identified, approved, and promoted through deployment environments.

```
Approach A:
    source control -> target machine -> local build -> local deploy -> runtime

Approach B:
    source control -> controlled build -> artifact store -> deployment -> runtime config ->
    runtime
```

Approach A can work, especially for local development, prototypes, disposable test hosts, and small systems. The problem is not that it is logically impossible to control. The problem is that it puts too many responsibilities on each target machine. The target becomes a source checkout, dependency resolver, build host, deployment host, and runtime host at the same time.

Approach B makes the release boundary explicit. The production host receives a known deployable unit; it does not create the release by compiling source during deployment.

3 The Control Boundary

Artifact promotion separates responsibilities that are often collapsed in weaker deployment processes.

```
Source control      -> code history
Build system        -> artifact creation
Artifact store      -> release identity
Deployment process  -> controlled promotion
Runtime environment -> configuration and secrets
Production host     -> execution
```

This is the central pattern: source control is where code evolves; artifact promotion is how production change is controlled.

The build can run in Jenkins, GitHub Actions, GitLab CI, Azure Pipelines, AWS CodeBuild, a release workstation, or another controlled environment. The defining property is not the product. The defining property is that the deployable unit is built once, identified, and promoted.

4 Environment-Scoped Runtime Configuration

A stable deployment model also separates artifact identity from environment identity.

Source control should contain the same application code for every environment. The artifact should contain built application bits and deployment logic, but not environment-specific runtime data. That runtime data has two different categories. The first category is true secrets: production passwords, test passwords, SMTP credentials, database passwords, API tokens, private keys, OAuth secrets, and similar values. The second category is operational configuration: database connection strings, SMTP hosts, internal service URLs, bucket names, service identifiers, feature flags, and environment names. The first category must be protected as secret. The second category may not always be secret, but it still defines the environment in which the artifact runs. Both belong to the runtime environment, not to source control and not to the artifact.

A simple implementation is **environment-scoped runtime configuration**. For each environment, there is one protected configuration set. All instances in the same environment receive the same set. Creating a new environment, such as QA, PROD2, disaster recovery, or a temporary test replica, means provisioning a new configuration set; it does not require changing source code or rebuilding the artifact.

```
Same source code
Same artifact
Different runtime configuration per environment
```

The mechanism can be simple or sophisticated. A Linux service may read a protected file such as `/etc/<app>/system.properties`. A Windows service may read a protected configuration file from a service-owned directory. A container platform may mount configuration or inject environment variables. Larger platforms may use a parameter store, vault, key vault, managed identity, secret manager, or internal configuration service. Public endpoints can be documented openly when they are truly public. Internal service URLs and other environment coordinates are different: they are not necessarily secrets, but they are still operational configuration and should be supplied by the environment. The architectural rule is the same:

> Code is in the repository. Built application bits are in the artifact. Secrets and environment-specific operational configuration are supplied at runtime.

Approach A does not logically require secrets in source control. A disciplined team can still use runtime configuration correctly. The risk is structural pressure. If every target machine checks out source, selects build profiles, resolves dependencies, compiles, and deploys locally, the boundaries between source, build configuration, deployment configuration, and runtime configuration are easier to blur. Environment-specific files, branches, scripts, profiles, and local overrides tend to accumulate near the source checkout. Even when these files do not contain literal passwords, they increase the risk of secret sprawl and accidental commits.

Artifact promotion makes the cleaner boundary natural. The artifact does not know whether it is running in test, production, a replica environment, or a temporary environment. It starts on a prepared host and reads the configuration supplied there.

5 Operational Effects

Artifact promotion is not only packaging. It changes the failure modes of deployment.

- **Bit-identical promotion.** The same built bytes can be promoted through environments. Testers and business stakeholders approve the release that production later receives.
- **Tamper-evident provenance.** The question "What is deployed?" has a concrete answer: artifact name, version, checksum, source revision, build time, and approval record.
- **Atomic rollback.** Rollback means redeploying a known previous artifact. It does not mean reverting source and hoping every target machine rebuilds the same output.
- **Cleaner trunk discipline.** The environment is not encoded by a branch. The environment is encoded by runtime configuration supplied to the artifact.
- **Smaller production surface.** Production hosts do not need source trees, build tools, compiler toolchains, or dependency-resolution credentials merely to receive a release.
- **Restricted-network compatibility.** Production networks often restrict outbound access. Artifact promotion works naturally when production cannot reach public source-control or dependency services.

The corresponding failure modes of build-on-target-machine deployment are familiar to many production teams: toolchain drift, local dependency drift, partial-fleet deployment, environment-specific branches, copied profiles, source-control outages during deployment, and difficulty proving what was actually deployed.

6 Source-Control Compromise and Blast Radius

If source control is compromised, every deployment model requires incident response. Artifact promotion does not make malicious code impossible. An attacker who can alter source may try to affect a future build.

The difference is the control path. Under build-on-target-machine deployment, production machines may consume the compromised repository directly at deployment time. If deployment means `git pull` followed by a local build, source control is part of the production execution path.

Under artifact promotion, production deploys a previously produced artifact whose identity can be hashed, signed, scanned, approved, and archived. A source-control compromise can still affect a future build, but it does not by itself reveal runtime secrets, rewrite the identity of an already approved artifact, or force production to compile from the current repository state. Environment-specific operational configuration remains outside the repository as well, reducing accidental exposure of internal topology and deployment coordinates.

This is the main security property: not that source control can never be compromised, but that source control, artifact identity, deployment approval, and runtime secrets are separate control domains.

7 Regulated and High-Assurance Environments

In regulated and high-assurance environments, the issue is not only whether the software works. The organization must often prove what was approved, what was deployed, who approved it, when it changed, whether configuration was controlled, and whether the release can be reproduced or rolled back.

Artifact promotion supports that evidence because the release is a stable object. The artifact can carry or be associated with version, checksum, source revision, build time, approver, scan result, deployment time, and rollback history. Build-on-target-machine deployment makes the evidence harder to establish because each target machine may create its own build output at deployment time.

This matters in healthcare, finance, federal, defense, payment, aviation, and life-science contexts. Examples of relevant regulatory, certification, and assurance vocabularies include HIPAA, HITRUST, SOC 2, SOX, FFIEC, PCI DSS, FedRAMP, FISMA, NIST SP 800-53, DoD Impact Levels IL4/IL5/IL6, CMMC, NIST SP 800-171, DFARS cybersecurity clauses, FDA 21 CFR Part 11, GxP practices, and DO-178C. These frameworks do not all prescribe the same technical pipeline, and artifact promotion alone does not guarantee compliance. But they all increase the value of release identity, segregation of duties, configuration control, integrity verification, and auditable deployment history.

For project managers and technical leads, the practical guidance is simple: do not manage releases as ad-hoc builds on machines. Manage releases as approved artifacts.

8 Implementation Examples

In Java, a controlled build may use Ant, Maven, or Gradle to produce a jar, war, zip, or container image. In .NET, it may use MSBuild or `dotnet publish` to produce a service package, published directory, installer, or container image. In Windows estates, the deployment unit may be an installer, service bundle, PowerShell-delivered package, or deployment archive. In container platforms, the artifact is often an image promoted through a registry.

On AWS, artifacts may be stored in S3 or a container registry and deployed to EC2, ECS, EKS, Lambda, or another runtime through deployment agents, SSM Run Command, Auto Scaling Group instance refresh, CodeDeploy, or platform-specific rollout mechanisms. On Azure, pipeline artifacts, release stages, deployment jobs, environment checks, and approvals implement the same general separation. The tools differ; the pattern does not.

A deployment procedure is still useful in Approach B. Its job is not to compile the application. Its job is to install or activate an already-built artifact: stop a service, unpack files, move a symlink, update a service directory, run platform steps, restart a process, and report success or failure.

Database migrations, one-time data loads, ETL jobs, and destructive operational steps should be controlled separately from automatic instance deployment. They should not re-run merely because a new application instance starts.

9 Common Objections

"Old artifacts ship vulnerabilities." Old artifacts should not be promoted blindly. Vulnerability response is cleaner under artifact promotion: rebuild once, identify the new artifact, scan it, approve it, and promote it. The alternative is coordinating source pulls and local builds across a fleet.

"This is overengineering." The mechanism can be simple. An artifact store, a versioned package, a deployment procedure, and an environment-scoped runtime configuration set are enough to establish the control boundary.

"Source on the server helps debugging." Production debugging should use logs, metrics, traces, symbols, source maps, source jars, or controlled diagnostic packages. Keeping a source checkout on production is not the same as having a stable release process.

"Approach A can also use runtime secrets." Correct. The claim is not that Approach A must store secrets in Git. The claim is that artifact promotion makes the separation easier to enforce and easier to audit, because source, artifact, deployment, and runtime configuration have different identities and owners.

10 Bottom Line

Build-on-target-machine deployment treats deployment as a repeatable build event performed on each target. Artifact promotion treats deployment as controlled movement of a known release.

Production systems need the second model. The mature release question is not "Did production build the latest code?" It is "Which approved artifact is deployed, and which environment-scoped runtime configuration is it using?"

That question is understandable to stakeholders, actionable for project managers, precise for engineers, and auditable for regulated environments. It is the difference between a deployment habit and a release-control model.